

增強課程 8：期末綜合實作——真實客戶需求案例

Lecture

en04 ~ en07 每一題都是「生產者視角」的深度練習：自己動手建立 Implicit Enhancement Point (en04)、自己建 Enhancement Spot / BAdI Definition (en05)、自己寫 Implementation (en06)、自己在 Z 程式裡宣告 Explicit Enhancement Point/Section (en07)。這些練習把每種技術的建立機制、排錯眉角都摸透了，但有一個關鍵盲點——一般 **ABAPer** 在真實專案裡，幾乎不會有機會「自己宣告」一個 **Explicit Enhancement Point**。原因很簡單：Z 程式是自己的，想改邏輯直接改程式碼就好，不需要刻意留一個「以後可能有人要插入程式碼」的位置；會主動宣告插入點的，通常是 SAP 自己（在標準物件裡）或套件開發商（賣框架給別人客製）。

真正日常會遇到的工作模式，反而是**消費者視角**：業務單位提出一個需求，需求剛好命中一個**標準物件**（不能碰它的原始碼），這時候要做的是——

- 1. 到標準物件裡**找**有哪些既有掛勾點（Classic User-Exit / Classic BAdI / New BAdI / Implicit Enhancement / Explicit Enhancement）
- 2. **判斷**該用哪一種（en01 已經練過這個判斷力，這題要在真實案例上再練一次）
- 3. 在那個掛勾點上**Implement**客製化邏輯
- 4. **驗證**沒有影響到標準流程本身

en08 就是照這個消費者視角，設計三個各自獨立的真實客戶需求，分別對應三種技術：

#	技術	案例	狀態
1	Classic User-Exit	PPCO0001：工單存檔時把 Component Change 存入 Log Table	✅ 已完整驗證
2	Implicit Enhancement	CO02/CO03 Cost Analysis 權限漏洞：標準功能不卡使用者權限，補上廠別層級授權檢查	✅ 已完整驗證
3	Explicit Enhancement (消費者視角)	MB52 加 MRP Controller 選取欄位，掛在 SAP 標準程式已經存在的 Explicit Enhancement Point 上	✅ 已完整驗證

案例 3 特別值得注意：跟 en07 剛好相反——en07 是「生產者視角」，自己在 Z 程式裡宣告插入點；這裡是**消費者視角**，SAP 標準程式（**RM07MLBS**，MB52 底層報表）裡已經存在別人（另一個開發者）留下的 **ENHANCEMENT-POINT**，這次要做的是在同一個插入點上再掛一個新的 **Implementation**，不用自己宣告任何東西。

案例二：CO02/CO03 Cost Analysis 權限漏洞 (Implicit Enhancement)

需求

使用者在 **CO02**（變更生產訂單）功能表裡做「Cost Analysis（成本分析）」，標準功能**不會檢查使用者權限**——任何登入的使用者都能看到任何工廠的成本資料，需要補上一個**依廠別（Plant）判斷的權限檢查**，沒有權限就跳出錯誤訊息擋下來。

排錯過程：怎麼找到真正的漏洞點

這是整題最花時間、也最有教學價值的部分——光是「猜」插入點在哪裡是不夠的，必須用除錯器實際追出真實呼叫鏈。

1. **TSTC** 查交易碼對應程式：**C002** → **SAPLCOK01** (跟 en04 用過的暖身呼叫 **CO_K01_GET_HEADER** 同一個 Function Group **COK01**)
2. **System→Status** 追蹤實際執行的程式：在 **C002** 開一張真實工單、點 Cost Analysis 之後，畫面顯示 **Program(GUI)=SAPLKKBC**——這是標準的 Report Painter/Report Writer 成本報表引擎 (跟交易碼 **KKBC_ORD** 是同一套)
3. ****ADT quickSearch K_KKB_*** 找候選進入點：找到 **K_KKB_CO_OBJECT_REPORT_CALL** (Function Group **KKB2**)，讀原始碼發現一個關鍵矛盾——函式裡真的寫了權限檢查 (**CALL FUNCTION 'G_JOB_AUTHORITY_CHECK' ... EXCEPTIONS NO_AUTHORITY = 1.**)，但這段檢查只存在於舊式 Report Painter 批次報表路徑；函式一開頭有個 **IF L_GRID = 'X'** 分支——這才是現代 ALV Grid 顯示路徑，呼叫 **K_KKB_CO_OBJECT_DISPLAY / K_KKB_KKBCS_ORDER_REPORT** 之後直接 **EXIT**，完全跳過**下面那段權限檢查
4. 在 **SE37** 對這支 **FM** 下中斷點，實際點 **CO02/CO03** 的 **Cost Analysis**，用除錯器逐步確認：
 - **L_GRID** 真的等於 **'X'**
 - 走的是 **l_global_object-aufnr** 有值的分支，呼叫 **K_KKB_KKBCS_ORDER_REPORT** (工單專屬報表)，緊接著就 **EXIT**。
 - 完整 Call Stack：**CO_COST_SHOW_ORDER...(SAPLCOCOST) → CK_F_CO_OBJECT_DISPLA...(SAPLKKCP) → CO_OBJECT_DISPLAY_WI...(SAPLCK10) → KKCK_CO_OBJECT_DISPL...(SAPLKKCK) → IF_CO_PC_CO_OBJECT~D...(Method) → K_KKB_CO_OBJECT_REPORT_CALL(SAPLKKB2)**

這一步證實了漏洞的真正成因：SAP 標準程式碼裡確實寫了權限檢查，但只覆蓋到舊路徑；現在實際在用的 ALV Grid 路徑是一塊真空地帶。這不是「SAP 忘記寫權限檢查」，而是新舊兩條路徑並存、新路徑漏補了同一段檢查——跟 en01 學到的「新舊技術並存、不是誰取代誰」是同一種系統演進現象，只是這次演進留下了一個安全缺口。

為什麼不能直接插在函式中間

en04 已經學過：Implicit Enhancement Point 只存在於 **FORM / FUNCTION / METHOD / MODULE** 的開頭或結尾，不能插在函式中間 (像 **IF L_GRID = 'X'** 分支裡面)。所以正確的插入點是 **K_KKB_CO_OBJECT_REPORT_CALL** 函式的最開頭——這樣不管走 Grid 還是舊路徑都能一併攔到，設計也更乾淨。

設計與建置

權限物件：標準 **K_ORDER** 欄位是 **RESPAREA / AUFART / AUTHPHASE / CO_ACTION / KSTAR**，沒有 **WERKS**，用不上，改自建 Z 權限物件：

- **SU21** 建立 **Z_EN08_COS** (10 碼上限，Object Class **CO**)，欄位 **ACTVT + WERKS** (皆為既有標準欄位，不用新建)
- 額外要點「Permitted Activities」按鈕維護 **ACTVT** 允許的值 (**03 = Display**)
- **SU21** 完全沒有 **ADT** 路徑 (連 quickSearch 對已知存在的標準物件 **K_ORDER** 都回空結果，比 Search Help / T-code 更徹底)，只能 GUI 建立

Message Class：**ZEN08**，訊息 **001**

- 空殼可以用 `ADT POST /sap/bc/adt/messageclass` 建立且真的生效
- ⚠️ 但訊息文字寫入 **ADT** 完全失敗：正確的 XML schema 是 `mc:message` 直接當 `mc:messageClass` 的子元素（不包一層 `mc:messages`），屬性 `mc:msgno / mc:msgtext`；PUT 回 200 OK，但讀回來還是空的，連補一次 Activation 都救不回來——這是本課程第二次遇到「回應成功但資料沒有真的落地」（第一次是 en01 ~ en07 課程外，Table Type 用錯 Content-Type 的案例），這次確認 schema / Content-Type 都對，判斷是這個環境的真實功能缺陷
- 最終文字只能請使用者到 SE91 手動補：`Plant &1 has no authorization for cost analysis`（後來依使用者建議改成更自然的措辭：「沒有權限顯示廠別 &1 的成本分析」——原本的英文措辭把「沒有權限」講成廠別的屬性，容易讓人誤讀成廠別本身沒有權限，而不是使用者沒有權限）

插入點：`K_KKB_CO_OBJECT_REPORT_CALL` 函式最開頭（SE37 → Edit → Enhancement Operations → Show Implicit Enhancement Options → Start of Function → Create Implementation，跟 en04 手法一致）

踩坑：**KOPF-WERKS** 抓不到值：第一版程式碼直接用函式的匯入參數 `KOPF LIKE CKI64A OPTIONAL` 裡的 **KOPF-WERKS** 判斷廠別，實測發現訊息跳出來時廠別是空白（`Plant has no authorization...`）。用除錯器檢查發現 **KOPF-WERKS** 真的是空的——原因是這個欄位只有在「用 **KOKRS** 反推廠別」那個分支才會被用到，**工單型的呼叫路徑（AUFNR 有值）** 完全不會去碰它，**CKI64A** 這個結構本質上是給「成本估算畫面」用的輔助欄位，不是工單專屬結構。

修正：動態 **ASSIGN** 讀取跨程式全域變數：改用 en04 第 23 節學過的技巧，直接指向 **SAPLCOK01** 全域的 **CAUFVD**（工單抬頭工作區）：

```
FIELD-SYMBOLS: <fs_werks> TYPE any.
DATA: lv_werks TYPE werks_d.

ASSIGN ('(SAPLCOK01)CAUFVD-WERKS') TO <fs_werks>.
IF <fs_werks> IS ASSIGNED.
    lv_werks = <fs_werks>.
ENDIF.

AUTHORITY-CHECK OBJECT 'Z_EN08_COS'
    ID 'ACTVT' FIELD '03'
    ID 'WERKS' FIELD lv_werks.
IF sy-subrc <> 0.
    MESSAGE e001(zen08) WITH lv_werks.
ENDIF.
```

因為這次是真的從 **CO02/CO03** 交易一路呼叫下來（不像 en04 的獨立測試程式需要先做一次「暖身呼叫」讓目標程式載入記憶體），**SAPLCOK01** 這時候必然已經在記憶體裡，所以不需要暖身呼叫，**ASSIGN** 直接就成功了。

驗證結果

使用者親自用真實工單（**Plant 1011**）在 **CO02/CO03** 測試兩種情境，完整端對端驗證成功：

- 有 **Z_EN08_COS** 權限：正常進入 Cost Analysis 報表（Target/Actual Comparison，顯示真實成本資料）

- 沒有 **Z_EN08_COS** 權限：畫面狀態列跳出「沒有權限顯示廠別 1011 的成本分析」，畫面被正確擋下，無法看到成本資料

案例一：PPCO0001 (工單存檔時 Component Change 存 Log Table)

——  已完整驗證

需求：工單存檔時，把使用者對工單元件 (Component) 的異動內容 (改前/改後值) 記錄進自訂 Log 表，方便事後稽核。

已查證的技術背景：

- Enhancement 名稱 **PPCO0001**，Function Exit 是 **EXIT_SAPLCOBT_001** (Function Group **XC01**，Include **ZXC01U01**) ——簽章 **COMPONENT_TABLE LIKE RESBB / HEADER_TABLE LIKE CAUFVDB**，剛好對得上參考文件的比對邏輯需要的參數
- 查 **MODACT** 表：本系統 0 筆，從未被任何 **CMOD Project** 指派過，是乾淨的教學素材
- 使用者提供完整的真實參考文件 (工單存檔時將 **Component Change** 存入 **Log Table.docx**)，內含實際運作過的實作邏輯：核心是 **FORM log_co02_change_content**，逐欄位比對元件變更前後的值 (12 個比對欄位：
POSNR/MATNR/ERFMG/ERFME/WERKS/LGORT/CHARG/SCHGT/RGEKZ/SOBKZ/KZEAR/XLOEK)，寫入自訂 Log 表，過程含 **ENQUEUE/DEQUEUE** 鎖定與使用者/時間戳記錄
- 文件裡的第二份欄位清單 (**CDCHNGIND/ZC002LOG_SEQ/FIELDNAME/CDFLDVALO/CDFLDVALN/UNAME**) 比對後發現是各欄位對應的 **Data Element**——**CDCHNGIND/FIELDNAME/CDFLDVALO/CDFLDVALN/UNAME** 剛好是 SAP 標準異動文件 (**CDHDR/CDPOS**) 在用的標準 DE，直接沿用比自建更合理

設計與建置 (承 CLAUDE.md 風格規範「要用 Data Element，不要直接給 Type」)：

- **ZEN08_COMPLOG (Log 表，ADT 建立)**：17 個欄位全部用 Data Element，沿用標準 DE (**AUFNR/ARTNR/UPDAT/UPTIM/RSNUM/RSPOS/RSART/CDCHNGIND/FIELDNAME/CDFLDVALO/CDFLDVALN/UNAME/TCODE/TEXT30/TEXT80**)，唯一沒有語意相符標準 DE 的序號欄位 (**SEQ**) 自建 **ZEN08_SEQ** (Domain+DE，NUMC 3 碼)
- **EZEN08_COMPLOG (Lock Object，使用者於 SE11 建立)**：Primary Table **ZEN08_COMPLOG**，Lock Mode **E (Exclusive/Write Lock)**，Lock Parameter 勾選 **MANDT/AUFNR/ARTNR** (對應參考文件 **ENQUEUE_EZPPT005** 用的 **mode_zppt005 = 'E'**，鎖工單+成品兩個欄位) ——建好後系統自動產生 **ENQUEUE_EZEN08_COMPLOG/DEQUEUE_EZEN08_COMPLOG** 兩個 FM
- **ZCL_EN08_COMPLOG (Z Class，ADT 建立)**：把參考文件的比對/寫入邏輯搬進 **LOG_CHANGE** 靜態方法，呼應 en07 思考題 3 提過的「把邏輯集中寫在 Z Class，Enhancement 裡只放一行呼叫」設計
- **CMOD (使用者執行，比照 en02 四步驟)**：Project 建立 → Assign Enhancement **PPCO0001** → 雙擊 **EXIT_SAPLCOBT_001** 生成 Include **ZXC01U01** → 內容只放一行 **CALL METHOD zcl_en08_complog=>log_change(it_head = header_table it_comp = component_table)**。 → Activate Project

Lock Object 的 Lock Mode 選項差異 (SE11 建立時必填，四個選項)：

Lock Mode	說明	適用情境
E (Exclusive/Write Lock)	同一時間只能有一個人鎖定；同一個使用者可以重複呼叫 ENQUEUE 疊加鎖定次	本題選這個 ——確保同一張工單的元件異動記錄，同一時間只有一人在寫

Lock Mode	說明	適用情境
	數，但要呼叫相同次數的 DEQUEUE 才會真正解鎖	入，避免兩個人同時改同一張工單、記錄互相覆蓋或遺失
S (Shared/Read Lock)	多人可以 同時 持有 S 鎖 (唯讀情境)，但只要有人持有 S 鎖，其他人就拿不到 E 鎖	適合「允許多人同時讀取、但寫入要互斥」的情境
X (Exclusive，不可累加)	效果類似 E，但 同一個使用者 重複呼叫 ENQUEUE 會直接失敗 (不像 E 允許同一人疊加鎖定次數)	適合「絕對只能鎖一次，連自己都不能重入」的嚴格情境
O (Optimistic)	不是真的鎖資料，只記錄狀態供之後比對是否被別人改過 (樂觀鎖定)	適合長時間編輯、不想整段時間占著鎖的情境

⚠ **排錯記錄：source-based Class 存檔失敗，錯誤訊息完全文不對題——建立 ZCL_EN08_COMPLOG 時反覆遇到 ExceptionResourceScanDuringSaveFailure (訊息宣稱「類別無法拆分成 SECTION」)，但實際上跟 SECTION 完全無關，逐步排除後找到兩個真正的根因：**

1. **PROTECTED SECTION**。這個區塊即使空白也必須明確存在——中間拿掉整段 **PROTECTED SECTION**。(讓 **PUBLIC SECTION** 後面直接接 **PRIVATE SECTION**)，會讓這個 source-based 存檔的解析器整個崩潰，回這個誤導性的「無法拆分 SECTION」錯誤，實際上只是少了一個空區塊。
2. **CLASS-METHODS** 的 **IMPORTING / RETURNING** 參數，型別不能用 **TYPE STANDARD TABLE OF <結構>** 這種 **inline** 寫法——這種寫法只能用在區域變數的 **DATA/TYPES** 宣告，用在 Method 的正式參數宣告上會讓同一個解析器崩潰、報出同樣誤導性的錯誤。正確做法：Method 簽章維持泛型 **TYPE STANDARD TABLE** (不指定 **OF**)，方法內部改用具名 **TYPES** 定義表格型別，再讓 **FIELD-SYMBOLS** 參照那個型別名稱 (**FIELD-SYMBOLS** 同樣不支援 inline 的 **OF <結構>** 寫法)，最後用一般 **ASSIGN it_head TO <lt_head>.** (不需要 **CASTING**) 取得靜態型別化的存取，才能同時滿足 Method 簽章的限制、又讓 **SELECT ... FOR ALL ENTRIES** 這類需要編譯期靜態型別的語句正常運作。

⚠ **排錯記錄：Client 欄位一開始沒有跟標準命名慣例，導致真實 CO02 測試直接 Dump——ZEN08_COMPLOG 表格建立時，Client 欄位第一版取名 CLIENT (型別 abap.clnt)，語法上完全合法、也能正常啟用；但 SE11 建 Lock Object 時，系統會依欄位名稱自動產生 Enqueue/Dequeue FM 的參數名稱，於是 ENQUEUE_EZEN08_COMPLOG/DEQUEUE_EZEN08_COMPLOG 的參數也叫 CLIENT，不是慣例的 MANDT。真實在 CO02 觸發時，ZCL_EN08_COMPLOG 呼叫這兩個 FM 時寫的是慣例寫法 **EXPORTING mandt = sy-mandt**，結果 Dump：CALL_FUNCTION_PARM_UNKNOWN / CX_SY_DYN_CALL_PARAM_NOT_FOUND，「Function parameter 'MANDT' is unknown」。標準表格的 Client 欄位一律要叫 MANDT (型別引用標準 Data Element MANDT)，這是貫穿整個 SAP 系統的強制慣例，不是「隨便取名、程式配合欄位名稱就好」(第一次修正時我一度改成讓程式配合欄位名稱 **client = sy-mandt**，方向反了，應該是欄位名稱要配合慣例)。修法：把表格欄位改回 **key mandt : mandt not null;**，但這會連動觸發 "ZEN08_COMPLOG-CLIENT is used in aggregate EZEN08_COMPLOG. Delete not allowed" (Lock Object 還在參照舊欄位)——要先到 SE11 把 Lock Object 的 Primary Table 移除再重新加回 (讓欄位清單重新抓到 MANDT)，勾選回 MANDT/AUFNR/ARTNR 並存檔，才能讓表格的欄位改名生效；表格與 Class 都改回 **mandt** 之後重新啟用即修復。**

✅ **端對端真實驗證結果 (SE16 查 ZEN08_COMPLOG)：**

欄位	實際值
Client	130
Order	1100000000006
Product	SFG101
Changed on / At	2026/07/30 21:41:08
Reservation / Item no.	2577 / 5
Change Ind.	U (Update)
Field Name	ERFMG (Requirement Qty)
Old value → New value	100 → 120
User Name	MONICA
Transaction Code	CO02

真實在 CO02 把工單 **1100000000006** 的元件 **SFG101** 需求量從 100 改成 120 並存檔後，Log 表正確寫入一筆異動記錄，欄位標籤 (Field Name 說明文字、User Name 全名) 也都正確帶出——證實 MANDT 命名修正後，**ENQUEUE_EZEN08_COMPLOG/DEQUEUE_EZEN08_COMPLOG** 鎖定、**ZCL_EN08_COMPLOG=>LOG_CHANGE** 比對寫入、CMOD Enhancement 派送三段完全打通，案例一至此**完整驗證通過**。

案例三：MB52 加 MRP Controller 選取欄位 (Explicit Enhancement，消費者視角) —— 已完整驗證

需求：**MB52** (顯示倉庫庫存) 的選取畫面，除了既有的 Purchasing Group 之外，再加一個 MRP Controller (**MARC-DISPO**) 選取欄位，並且這個欄位要真的用來篩選查詢結果，不能只是「畫面上多一個欄位、實際沒作用」。

跟 **en07** 的關鍵差異：en07 是「生產者視角」，**ENHANCEMENT-POINT/ENHANCEMENT-SECTION** 那一行本身是自己宣告的；這裡是「消費者視角」——插入點已經存在於標準程式裡 (別人留的)，這次要做的只有 **Create Implementation**，不需要 (也不能) 碰宣告本身。

一支程式裡的多個 Explicit Enhancement Point，怎麼對應到 Report Event

MB52 底層程式是 **RM07MLBS**，查證這套訓練系統 (非只有 2014 年參考文件那套舊系統) 發現這支程式裡同時留了幾十個 **ENHANCEMENT-POINT/ENHANCEMENT-SECTION**，但不是隨便亂灑——位置都對應到經典 **ABAP Report** 的事件 (**Event**) 結構：

Report Event 階段	這個階段的用途	RM07MLBS 裡對應的插入點 (舉例)
選取畫面宣告區 (任何 Event 之前)	宣告 PARAMETERS / SELECT-OPTIONS	ENHANCEMENT-POINT rm07mlbs_3 SPOTS es_rm07mlbs STATIC 。——卡在 Block "lbs" 裡，緊接在既有 SELECT-OPTIONS 宣告之後，本身也是 STATIC (只能放宣告式陳述式)，語意上就是「這裡可以再宣告新的選取欄位」

Report Event 階段	這個階段的用途	RM07MLBS 裡對應的插入點 (舉例)
內部資料結構宣告 (不屬於任何 Event，但也是宣告區)	讓別人在既有的內部工作結構裡加欄位	ENHANCEMENT-POINT ehp604_rm07mlbs_03 SPOTS es_rm07mlbs STATIC .——卡在 DATA: BEGIN OF bestand ... END OF bestand. 中間，真實案例 DIPCS_RM07MLBS001 就是在這裡加了 owner/lifnr2 兩個欄位到別的結構，用途是「擴充內部結構」，跟選取畫面完全無關
INITIALIZATION (選取畫面顯示前跑一次)	設定預設值、動態調整選取畫面文字/屬性	ENHANCEMENT-POINT rm07mlbs_07 SPOTS es_rm07mlbs. (非 STATIC，可執行程式碼)——緊接在 INITIALIZATION. 事件裡既有的 PERFORM / MOVE 邏輯之後、START-OF-SELECTION. 之前，真實案例 AD_MPN_MD_RM07MLBS 就是在這裡呼叫 SELECTION_TEXTS_MODIFY_DTEL 動態設定欄位文字
主要資料撈取邏輯 (START-OF-SELECTION 之後的處理常式內)	修改實際查詢邏輯	ENHANCEMENT-SECTION ehp604_rm07mlbs_08 SPOTS es_rm07mlbs .——包住主要的 SELECT ... FROM mara INNER JOIN ... marc 查詢邏輯本身，真實案例 MGVLAMA_RM07MLBS 在這裡把 mfrpn IN @mfrpn 加進 WHERE 子句

這個對照給出一個通用的搜尋策略：下次遇到「要在標準程式裡加東西」的需求，可以先判斷「我要加的東西屬於 Report Event 的哪個階段」——加選取欄位就去找宣告區的插入點；要動預設值/畫面文字就去找 INITIALIZATION / AT SELECTION-SCREEN 附近的插入點；要動篩選邏輯就去找 START-OF-SELECTION 之後、主要處理常式裡的插入點——不用漫無目的地整支程式找，鎖定對應的 Event 區塊就能大幅縮小範圍。這也解釋了為什麼同一個 Enhancement Spot (ES_RM07MLBS) 底下可以掛幾十個 Enhancement Option：Spot 是「這支程式對外開放客製化」的單一容器，Option 才是散落在程式各個 Event 階段裡、各自服務不同目的的實際插入點——這跟 en01 補充過的 Enhancement Spot / Option / Implementation 三層架構完全吻合，RM07MLBS 就是這個架構在真實標準程式裡的具體案例。

完整設計：三個插入點缺一不可

只加選取畫面欄位 (宣告) 而不動篩選邏輯，欄位會出現在畫面上但完全不起作用；只加篩選邏輯而不宣告欄位，程式會編譯失敗 (找不到變數)。三塊分工如下：

① rm07mlbs_3 (STATIC，選取畫面宣告區)：

```
SELECT-OPTIONS: dispo FOR marc-dispo.
```

② rm07mlbs_07 (INITIALIZATION 事件內，非 STATIC，設定選取畫面文字)：

```
DATA: lt_seltexts LIKE rsseltexts OCCURS 0 WITH HEADER LINE.
lt_seltexts-name = 'DISPO'.
lt_seltexts-kind = 'S'.
lt_seltexts-text = 'MRP Controller'.
APPEND lt_seltexts.
CALL FUNCTION 'SELECTION_TEXTS_MODIFY'
EXPORTING
```

```
program = sy-repid
TABLES
seltexts = lt_seltexts.
```

③ **ehp604_rm07mlbs_08** (**ENHANCEMENT-SECTION** , 主查詢邏輯, 整段取代) : 這套系統因為有 CWM (Catch Weight Management) 功能, 原邏輯是 **IF/ELSE** 兩個分支 (各自對應不同的 **SELECT** 語法) , 比 2014 年參考文件單純的版本複雜; 取代時**完整保留兩個分支的所有既有欄位與既有 **WHERE** 條件**, 只在各自的 **WHERE** 子句多加一行 **AND dispo IN @dispo** (詳細內容見下方「事前準備」) 。

建置結果與驗證

使用者在 SE38 建立一個 Enhancement Implementation **ZMB52_EE** , 裡面包含三個 **ENHANCEMENT n** 區塊, 分別對應上面①②③三個插入點——這跟一開始設想的「拆成三個獨立 Implementation 物件」不同, 但同樣合法: 一個 **Enhancement Implementation** 物件內部可以包含多個 **ENHANCEMENT n ... ENDENHANCEMENT** 區塊, 適用在「這一個客製化需求剛好要動好幾個插入點」的情境, 比拆成三個各自獨立的物件更能表達「這是同一件事」的語意。

⚠ **ADT** 讀取這類已被 **Explicit Enhancement** 過的標準程式, **sap_get_source** 讀到的內容可能是舊的 (快取/延遲問題) : 使用者已在 SE38 Activate 成功、SE38 Global Search 也能查到 **ZMB52_EE** 的三個區塊, 但同一時間用 ADT **sap_get_source** 重新讀取 **RM07MLBS** 的合併原始碼, 完全看不到任何新增痕跡 (沒有 **DISPO**、沒有新的 **SELECTION_TEXTS_MODIFY** 呼叫、插入點附近也沒有 "{ Begin ENHO ... }" 標記) 。這跟第 10 節記載的「Data Preview 快取舊 nametab」是同一類現象——**ADT** 對這類物件的讀取有自己的快取層, 不能只憑 **ADT** 讀不到就判斷「使用者的操作沒有真的生效」, 真正的驗證要靠使用者在 SAP GUI 實際操作 (畫面 / SE38 Global Search / 執行結果) 為準。

真實 GUI 端對端驗證, 三個插入點全部確認生效:

- 選取畫面正確出現新的「MRP Controller」欄位 (①②驗證成功: 欄位存在且文字正確顯示, 不是技術欄位名)
- 填入實際存在的 MRP Controller 代碼 (**PM1**) 執行查詢, 結果全部都是 **MRP控制員=PM1** 的料號, 證實 **WHERE** 子句真的加了篩選條件, 不是「畫面有欄位、輸入什麼都不影響結果」 (③驗證成功)

⚠ **意外發現**: 同一個插入點上有殘留的重複物件: 測試過程中發現選取畫面同時出現兩個「**MRP Controller**」欄位, 追查後確認是使用者先前自己測試時留下的 **ZRM07MLBS3** (宣告 **SELECT-OPTIONS: s_dispo for marc-dispo.**, 跟 **ZMB52_EE** 宣告的 **dispo** 是同一個底層欄位、不同變數名) , 而且這個殘留物件也是 **Active** 的, 代表兩個欄位同時生效中。這跟本課程一貫的「**\$TMP** 訓練物件留著也無妨」不同——這裡是掛在 **MB52** 這種大量使用者共用的真實標準交易上, 殘留的重複 **Implementation** 會直接影響所有正在用這支交易的人, 不是只影響自己的訓練環境, 已提醒使用者盡快到 SE38/SE19 清理掉 **ZRM07MLBS3**。這是一個很好的提醒: 在自建的 **\$TMP** 練習物件跟「掛在真實共用標準物件上的客製化」之間, 殘留物的清理急迫性完全不同等級, 後者要當作正式維運物件對待。

學習目標

- 能講出「生產者視角」 (en07: 自己宣告插入點) 跟「消費者視角」 (en08: 找 SAP 已留好的插入點) 的差異, 並知道真實工作裡消費者視角才是常態
- 能示範完整的「消費者視角」排錯流程: **TSTC** 查程式 → **System→Status** 追實際執行路徑 → **quickSearch** 找候選 FM → 讀原始碼判斷邏輯缺口 → 除錯器下中斷點實際驗證

- 能解釋為什麼「標準程式裡寫了權限檢查」不代表「所有執行路徑都會走到這段檢查」——新舊路徑並存時，舊路徑的檢查不會自動套用到新路徑
- 知道 Implicit Enhancement Point 只能插在 FORM/FUNCTION/METHOD/MODULE 的開頭或結尾，設計插入點位置時要往「函式最外層」找，不能想插在流程中間
- 能講出動態 ASSIGN 讀取跨程式全域變數的技巧，並判斷「當下情境是否需要暖身呼叫」（真實交易鏈觸發 vs 獨立測試程式）
- 知道 SU21（權限物件）完全沒有 ADT 路徑，是本課程遇過最徹底的 GUI-only 限制之一
- 知道 Message Class 的空殼可以用 ADT 建立，但訊息文字目前這個環境無法透過 ADT 寫入，這是「回應成功但資料沒有真的落地」的又一個真實案例
- 能講出「消費者視角找到既有 Explicit Enhancement Point」跟「生產者視角自己宣告」在操作上的差異：前者只需要 Create Implementation，不用碰宣告本身
- 能講出一支標準程式裡的多個 Explicit Enhancement Point，位置通常對應到 ABAP Report Event 的不同階段（宣告區 / **INITIALIZATION** / **START-OF-SELECTION** 之後），找插入點時可以先判斷「我要加的東西屬於哪個階段」來縮小搜尋範圍
- 知道一個 Enhancement Implementation 物件可以包含多個 **ENHANCEMENT** n 區塊，適合「一個客製化需求要動好幾個插入點」的情境
- 知道 **sap_get_source** / ADT 讀取已被 Explicit Enhancement 過的標準程式時，內容可能有快取延遲，不能只憑「ADT 讀不到」就判斷使用者操作沒有生效，要以 GUI 實際操作結果為準
- 能講出「掛在 \$TMP 訓練物件上的殘留」跟「掛在真實共用標準物件（如 **MB52**）上的殘留」在清理急迫性上的本質差異——後者會影響所有正在使用該標準交易的人

事前準備（已於本系統 client 130 實際完成，非假設）

1. **Z_EN08_COS**（權限物件，使用者於 **SU21** 建立）：Object Class **CO**，欄位 **ACTVT+WERKS**，Permitted Activities 已維護 **03**。
2. **ZEN08**（Message Class，ADT 建立空殼成功；訊息 **001** 文字使用者於 **SE91** 補上）：「沒有權限顯示廠別 &1 的成本分析」。
3. **ZIM_CO_COST_AUTH**（Implicit Enhancement Implementation，使用者於 **SE37** 建立，掛在 **K_KKB_CO_OBJECT_REPORT_CALL** 函式最開頭）：內容見上方案例二程式碼，已使用者親自用 **CO02/CO03** 真實工單（Plant 1011）雙重情境（有權限 / 無權限）端對端驗證成功。
4. **ZMB52_EE**（Explicit Enhancement Implementation，使用者於 **SE38** 建立，掛在 **RM07MLBS** 的三個既有插入點上：**rm07mlbs_3 / rm07mlbs_07 / ehp604_rm07mlbs_08**）：內容見上方案例三①②③程式碼，已使用者親自在 **MB52** 端對端驗證成功（選取欄位、畫面文字、篩選邏輯皆生效）。系統裡另有一個使用者先前測試留下的殘留物件 **ZRM07MLBS3**（同樣宣告指向 **MARC-DISPO**，變數名 **s_dispo**，已確認 Active），待使用者清理。
5. **ZEN08_COMPLOG/EZEN08_COMPLOG/ZCL_EN08_COMPLOG**（Log 表 + Lock Object + Z Class，ADT 建立 + 使用者於 **SE11** 建 Lock Object）+ **CMOD Project**（使用者建立，Assign **PPC00001**，Include **ZXC01U01**，已 Activate）：案例一完整鏈路，已使用者親自用 **CO02** 真實工單（**110000000006**，元件 **SFG101**）端對端驗證成功（見上方案例一驗證結果）。

題目需求

1. 畫出案例二的完整排錯流程圖：從「使用者反映 Cost Analysis 不卡權限」到「找到 **K_KKB_CO_OBJECT_REPORT_CALL** 函式開頭是正確插入點」，中間經過哪些工具 / 步驟？每一步驗證了什麼、排除了什麼可能性？

2. 解釋為什麼 **G_JOB_AUTHORITY_CHECK** 這段程式碼「存在」不代表「權限漏洞不存在」：這個現象背後反映了什麼樣的系統演進模式？你會怎麼跟不熟 ABAP 的專案經理解釋這個矛盾？
3. 解釋為什麼要先用 **KOPF-WERKS** 失敗、才改用 **ASSIGN ('(SAPLCOK01)CAUFVD-WERKS')**：這兩個資料來源的本質差異是什麼？如果一開始就去查 **CKI64A** 這個結構的用途說明，有沒有機會提早避開這個坑？
4. 比較案例二（**Implicit**）跟案例三（**Explicit**，消費者視角）的操作步驟差異：兩者都是要在標準程式裡「加東西」，為什麼案例二需要在 SE37 走「Show Implicit Enhancement Options」，案例三卻是直接對已存在的 **ENHANCEMENT-POINT** 做 Create Implementation？
5. 案例三為什麼需要三個插入點才能完成，缺任何一個會發生什麼事：如果只做①（宣告 **dispo**）不做③（修改 **SELECT**），畫面跟執行結果會是什麼樣子？如果只做③不做①，會發生什麼事？
6. **ZRM07MLBS3** 這個殘留物件的發現，給了什麼跟「物件清理」有關的通用提醒：對照 en05 提過的「留著命名過長、從未成功呼叫過的 **ZES_EN05_FLIGHT_GREETING** 當反面教材」，為什麼那個可以放著不管、**ZRM07MLBS3** 卻要盡快清理？兩者的關鍵差異是什麼？

參考答案

案例二完整排錯流程圖：

- ① TSTC 查 C002 對應程式
 - 確認底層是 SAPLCOK01（跟 en04 同一個 Function Group COK01）
 - ↓
- ② 實際點 Cost Analysis，System→Status 看 Program(GUI)
 - 確認真正在跑的是 SAPLKKBC（Report Painter/KKBC_ORD 引擎），排除了「Cost Analysis 走別的完全無關程式」的可能性
 - ↓
- ③ ADT quickSearch K_KKB_* 找候選 FM，讀原始碼
 - 找到 K_KKB_CO_OBJECT_REPORT_CALL，發現「有權限檢查、但只在舊路徑」的矛盾，排除了「SAP 完全沒寫過權限檢查」的假設
 - ↓
- ④ SE37 對該 FM 下中斷點，實際操作驗證
 - 確認 L_GRID='X'、確認真的走到 K_KKB_KKBCS_ORDER_REPORT 分支、確認緊接著 EXIT.、確認 Call Stack 完整路徑
 - 這一步排除了「①~③的推論是錯的」這個風險，是唯一的「真正執行證據」，前面三步都只是縮小懷疑範圍

「權限檢查存在≠沒有漏洞」的系統演進模式：SAP 標準程式碼往往不是一次寫成的，Report Painter 這套成本報表引擎有新舊兩種顯示路徑（傳統批次報表 vs 現代 ALV Grid），推測是 ALV Grid 路徑是後來才加上去的效能優化/使用者體驗改進，加的時候複製了大部分邏輯，但沒有連帶把權限檢查那段也複製過去——這在大型遺留系統的維護裡很常見：新舊功能路徑並存，新路徑補完既有邏輯時容易漏掉「不顯眼但重要」的部分（權限檢查通常不影響功能是否「看起來正常」，很容易被測試忽略）。跟專案經理解釋可以這樣說：「這不是 SAP 沒做過權限檢查，是系統升級時新增了一條更快的路，這條新路沒有把舊路上的安全閘一起搬過去，兩條路平常看起來結果一樣，只有『誰能看什麼』這件事不一樣」。

KOPF-WERKS vs CAUFVD-WERKS 的本質差異：**KOPF**（型別 **CKI64A**）是「顯示成本估算畫面的輔助欄位」，語意上服務的是物料成本估算（**Costing**）情境，不是「工單」情境——即使它剛好有 **WERKS** 欄位，也只在特定分支（用 **KOKRS** 反推廠別）才會被填值，工單導向的呼叫路徑完全不會經過那段邏輯。**CAUFVD** 才是真正的

「工單抬頭」全域工作區，任何工單相關畫面在載入時都會把它填好。如果一開始就用 **SE11** 或 **ADT** 查一下 **CKI64A** 的 Short Description (「Auxiliary Fields on Screens for Displaying Cost Estimates」)，就會看到它跟「工單」完全沒有直接關聯，能提早懷疑這個欄位在工單情境下不可靠——這是一個很好的提醒：拿到一個結構要用之前，先看一眼它的用途說明，不要只看「剛好有我要的欄位名稱」就直接用。

案例二 vs 案例三操作步驟差異：案例二要插入的位置 (**K_KKB_CO_OBJECT_REPORT_CALL** 函式最開頭) 是一個系統自動保證存在、但預設隱藏的 Implicit 插入點——任何 FORM/FUNCTION/METHOD 的頭尾都有，只是要切換「Show Implicit Enhancement Options」才看得到，因為它「本來就在那裡」，不需要任何人事先宣告。案例三要用的 **ENHANCEMENT-POINT RM07MLBS_3** 則是一個已經被某個開發者明確宣告過的 Explicit 插入點 (en07 教過這一步需要 Create Option)，這個宣告動作已經有人做過了 (做出 **mfrpn** 那個 Implementation 的人)，現在只是要在同一個位置加另一個 Implementation，所以只需要 Create Implementation，完全不用碰 Create Option 那一步——這正是 en07 (生產者視角，自己做 Create Option) 跟 en08 案例三 (消費者視角，別人已經做過 Create Option) 的操作步驟差異根源。

案例三三個插入點缺一不可的具體後果：只做① (宣告 **SELECT-OPTIONS: dispo FOR marc-dispo.**) 不做③——畫面上會出現一個技術欄位名 (沒有②的話標籤顯示不出「MRP Controller」，會是空白或系統預設的通用符號)，而且不管填什麼值查詢結果都不會改變，因為 WHERE 子句根本沒有引用 **dispo** 這個變數，等於一個「看起來有作用、實際上是裝飾品」的欄位，對使用者是更糟的體驗 (讓人誤以為篩選有生效)；只做③不做①——**ehp604_rm07mlbs_08** 裡的 **AND dispo IN @dispo** 會在編譯 / 啟用時直接報錯「Field DISPO is unknown」，因為 **dispo** 這個變數從未被宣告過，程式根本無法啟用，這種情況反而比「裝飾品欄位」更安全，因為錯誤會在啟用階段就被擋下來，不會流到使用者手上。

ZRM07MLBS3 vs ZES_EN05_FLIGHT_GREETING 清理急迫性的差異：en05 那個命名過長、從未成功呼叫過的 **ZES_EN05_FLIGHT_GREETING**，是一個獨立、**\$TMP** 的訓練練習物件——它存在與否，唯一會影響到的只有這門課自己的教材整潔度，沒有任何真實使用者的操作路徑會經過它，放著當反面教材完全沒有風險。

ZRM07MLBS3 則是掛在 **MB52** 這種大量使用者天天在用的真實標準交易上，而且已確認是 Active 狀態，代表它現在正在真實影響每一個開啟 **MB52** 選取畫面的人 (多一個重複、容易混淆的欄位)——兩者的關鍵差異不在於「物件本身好不好看」，而在於它有沒有掛在別人也會經過的真實執行路徑上：獨立的訓練殘留物只要沒人會碰到就無妨，掛在共用標準物件上的殘留物只要是 Active 的，就等於是正在生產環境裡運作的東西，清理急迫性完全不同等級。

思考題

1. 案例二的排錯過程用了四個步驟 (**TSTC / System→Status / quickSearch / 除錯器**)，如果只做到第三步 (讀原始碼判斷) 就直接動手寫 Enhancement，沒有做第四步的除錯器驗證，可能會踩到什麼風險？ (提示：原始碼裡的分支邏輯可能有好幾層 **IF**，光用眼睛讀不一定能百分之百確定「這次呼叫真的會走到我以為的那個分支」)
2. **SU21** 完全沒有 ADT 路徑、Message Class 訊息文字寫入 ADT 也失敗——這兩個限制加在一起，代表案例二這整個功能要正式上線，有哪些步驟是「Claude 完全幫不上忙，一定要人工在 GUI 完成」的？如果團隊要把這類「AI 協作但有 GUI-only 缺口」的工作流程寫成標準作業程序，你會怎麼分配「Claude 做什麼、人做什麼」？
3. 案例三已經實測驗證：使用者填 **PM1** 時查詢結果正確篩選，那如果使用者完全不填 MRP Controller 欄位 (保持空白)，**AND dispo IN @dispo** 這個條件會讓查詢結果變成「什麼都篩不到」還是「跟改之前完全一樣、不影響任何既有查詢」？這跟 en04 / en06 用過的「手動寫 **IF** 判斷特定條件才生效」的安全閘設計，是同一種機制嗎？ (提示：**SELECT-OPTIONS** 宣告的變數在使用者完全沒輸入時，是一個空的 range table，欄位 **IN** 空的 range table 在 ABAP 裡有特殊規則)